

Template lab

Trabalho de Laboratório — Padrão Template Method em Java

Sistema de Missões de Robôs Autônomos

Objetivo

Implementar, em Java, um pequeno sistema de robôs usando o padrão Template Method.

O objetivo é mostrar que uma superclasse pode definir a sequência geral de um algoritmo, deixando algumas etapas para as subclasses implementarem.

Contexto

Considere um sistema com diferentes tipos de robôs:

- RoboExplorador
- RoboSeguranca
- RoboEntrega
- RoboMedico

Todos herdam de uma classe abstrata chamada Robo.

Cada robô executa uma missão seguindo sempre a mesma sequência geral:

1. iniciar sistema
2. deslocar-se até o local
3. executar ação principal
4. comunicar resultado
5. retornar à base, se necessário

A sequência da missão deve ser definida apenas na superclasse Robo.

As subclasses devem especializar apenas algumas etapas da missão.

Estrutura esperada

Classe abstrata:

- Robo

Subclasses:

- RoboExplorador
- RoboSeguranca
- RoboEntrega
- RoboMedico

Parte 1 — Classe abstrata Robo

Implemente a classe abstrata Robo com o método final:

`executarMissao()`

Esse método deve representar o Template Method.

Ele deve executar, nesta ordem:

1. `iniciarSistema()`
2. `deslocar()`
3. `executarAcaoPrincipal()`
4. `comunicarResultado()`
5. se `deveRetornarBase()`, então `retornarBase()`

O método `executarMissao()` deve ser final, para impedir que as subclasses alterem a sequência geral da missão.

A classe Robo deve possuir:

Métodos concretos:

- `iniciarSistema()`
- `comunicarResultado()`
- `retornarBase()`

Métodos abstratos:

- `deslocar()`
- `executarAcaoPrincipal()`

Hook:

- `deveRetornarBase()`

O hook deve retornar true por padrão.

Parte 2 — Subclasses

Implemente as subclasses:

RoboExplorador

- desloca-se com esteiras
- explora o ambiente

RoboSeguranca

- desloca-se com rodas
- vigia a área

RoboEntrega

- desloca-se com rodas
- entrega pacote

RoboMedico

- desloca-se rapidamente
- presta socorro

Parte 3 — Uso do hook

Crie também uma subclasse:

RoboEntregaSemRetorno

Ela deve herdar de RoboEntrega e sobrescrever o hook:

`deveRetornarBase()`

Esse método deve retornar `false`.

Assim, o robô executa a missão, mas não retorna à base.

Parte 4 — Teste

Crie uma classe `TesteRobos`.

Ela deve:

1. Criar um vetor de robôs.
2. Percorrer o vetor.
3. Mostrar o nome da classe do robô.
4. Executar executarMissao() para cada robô.

Entrega

O aluno deverá entregar o Diagrama de classes em UML. O diagrama UML deve mostrar:

- Herança entre Robo e suas subclasses.
- Método final executarMissao().
- Métodos abstratos deslocar() e executarAcaoPrincipal().
- Hook deveRetornarBase().
- Métodos concretos da superclasse.

Perguntas adicionais:

1. Onde aparece a herança no projeto?
2. Qual método representa o Template Method?
3. Por que executarMissao() foi declarado como final?
4. Quais métodos são abstratos?
5. O que é o hook neste exemplo?
6. Qual subclasse usa o hook?
7. Por que este projeto é um exemplo do padrão Template Method?
8. Qual a diferença principal entre este exemplo e o padrão Strategy?

Restrições

- Não usar if ou switch para escolher o tipo de robô sempre que possível.
- A sequência da missão deve estar apenas em Robo.
- As subclasses não podem redefinir executarMissao().
- O código deve compilar e executar corretamente.
- Usar nomes coerentes e boa indentação.

Preparar o código que será apresentado no laboratório mas que não deve ser mandado no trabalho !